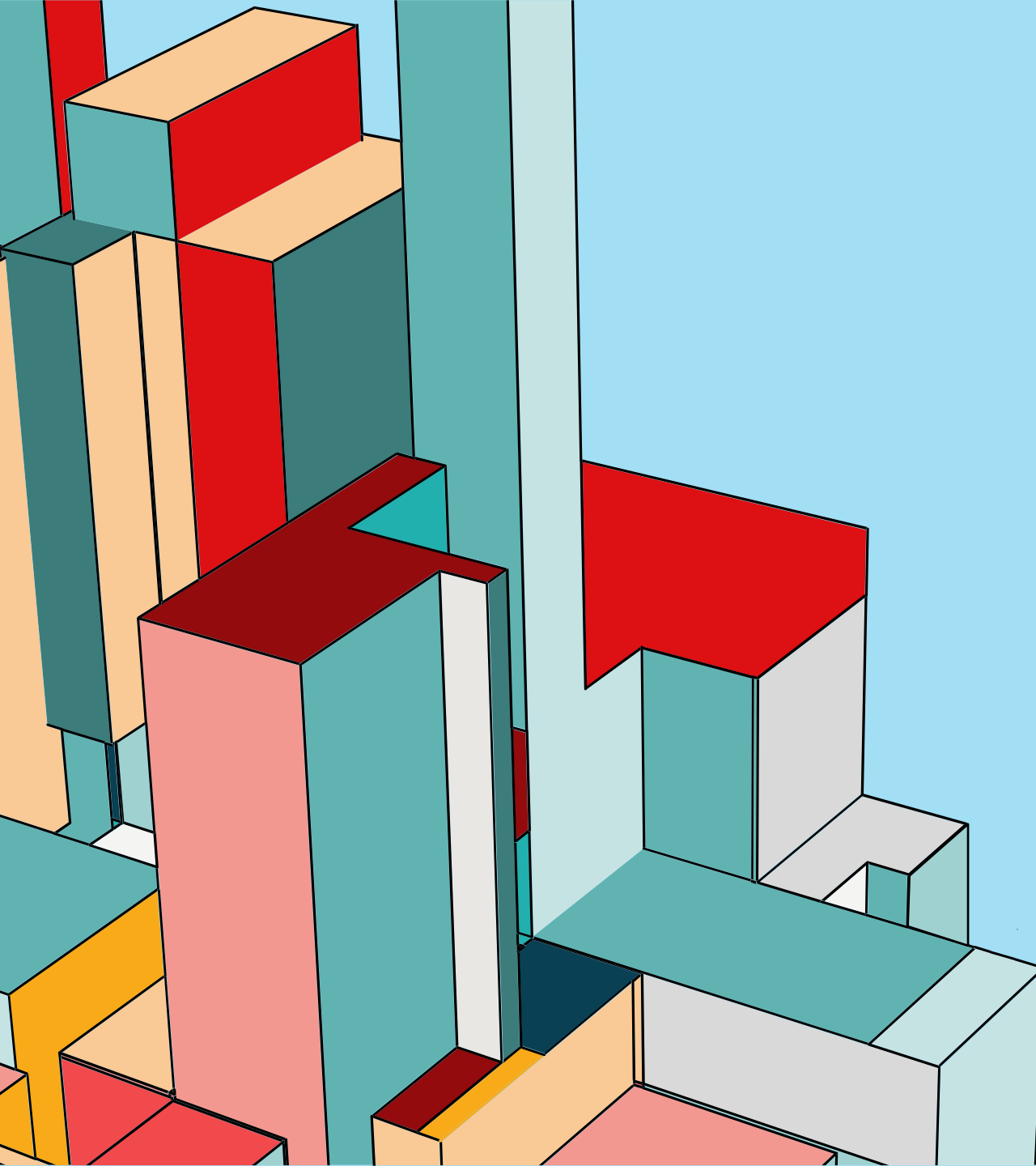


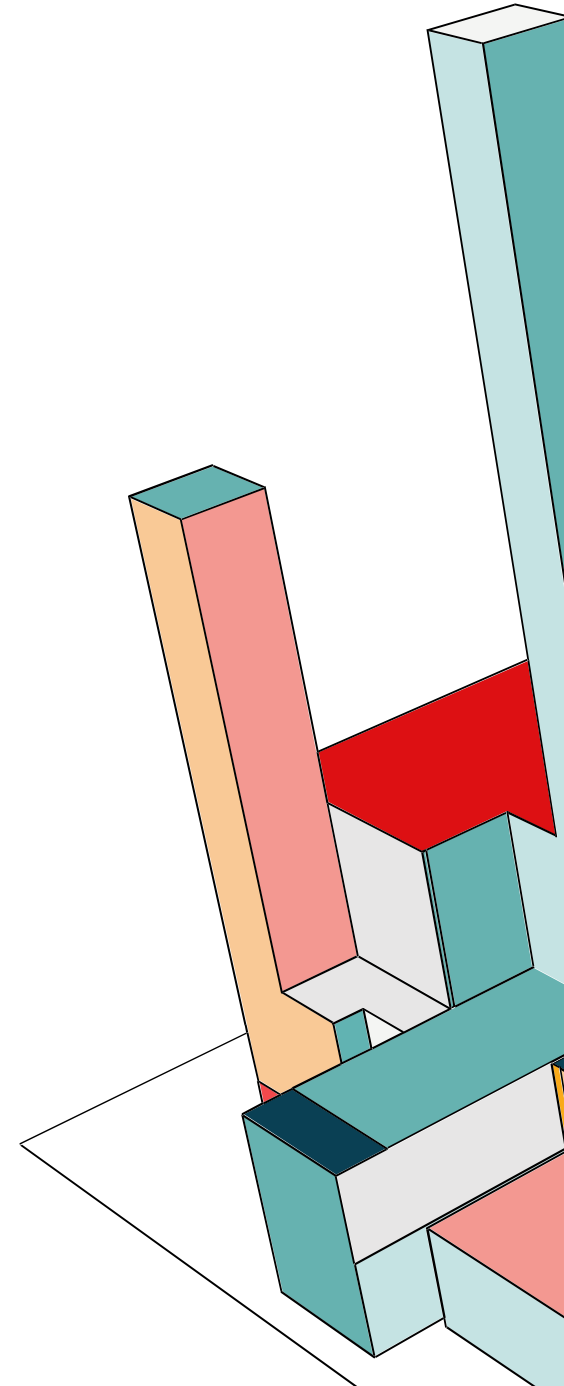


FUNCTIONS

CSCI 1380 Introduction to
Programming in Python

FUNCTIONS

- Function: group of statements within a program that perform as specific task
 - Usually one task of a large program
 - Functions can be executed in order to perform overall program task
 - Known as *divide and conquer* approach
- Modularized program: program wherein each task within the program is in its own function



DIVIDE & CONQUER

This program is one long, complex sequence of statements.

[illegible]

In this program the task has been divided into smaller tasks, each of which is performed by a separate function.

```
def function1():
    statement
    statement
    statement
```

function

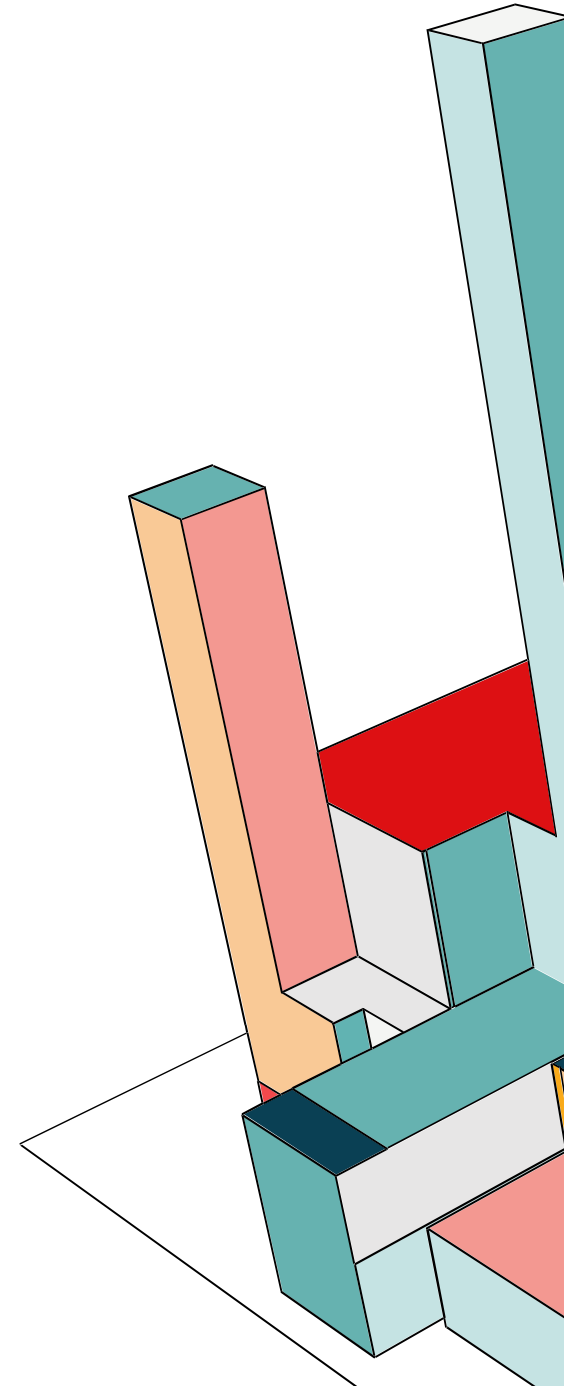
```
def function2():
    statement
    statement
    statement
```

function

```
def function3():  
    statement  
    statement  
    statement
```

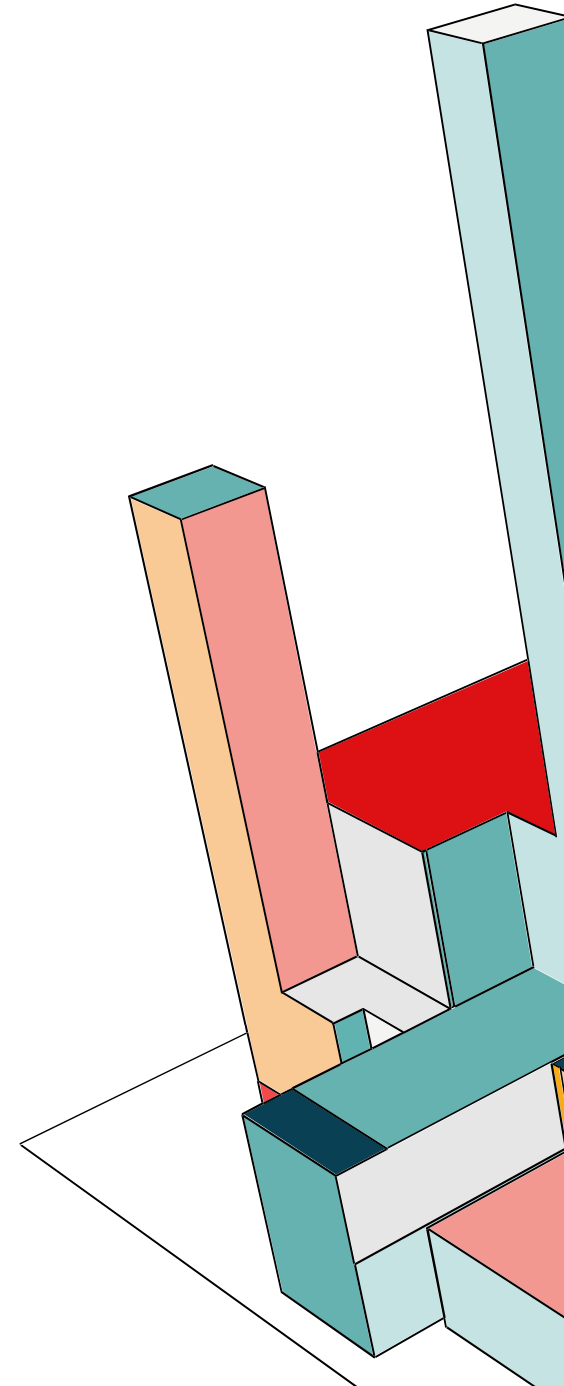
```
def function4():
    statement
    statement
    statement
```

function



BENEFITS OF USING FUNCTIONS

- The benefits of using functions include:
 - Simpler code
 - Code reuse
 - Write the code once and call it multiple times
 - Better testing & debugging
 - Can test and debug each function (find out what's wrong)
 - Faster development
 - Easier facilitation of teamwork
 - Different team members can write different functions.



DEFINING A FUNCTION

- Function name should be descriptive of the task carried out by the function.
 - Often includes a verb
- Function definition: specifies what function does
- Function header: first line of function
 - Includes keyword `def` and function name, followed by parenthesis and colon.
- Block: set of statements that belong together as group.

```
def function_name() :  
    statement  
    statement  
    ...
```



CALLING A FUNCTION

- Call a function to execute it.
 - When a function is called:
 - Interpreter jumps to the function and executes statements in the block.
 - Interpreter jumps back to part of program that called the function.
 - Known as function return

```
def function_name() :  
    statement  
    statement  
    . . .
```

```
function_name() # call
```



LOCAL VARIABLES

- Local variable: variable that is assigned a value inside a function.
 - Belongs to the function in which it was created
 - Only statements inside that function can access it, error will occur if another function tries to access the variable
- Scope: the part of a program in which a variable may be accessed.
 - For local variable: function in which created

```
def function_A() :  
    x = 10
```

```
def function_B() :  
    print( x ) # ERROR!
```

```
print( x ) # ERROR!
```



VOID FUNCTION & VALUE-RETURNING FUNCTIONS

- A void function:
 - Simply executes the statements it contains and then terminates.
- A value-returning function:
 - Executes the statements it contains, and then it returns a value back to the statement that called it.
 - The `input`, `int`, and `float` functions are examples of value-returning functions.

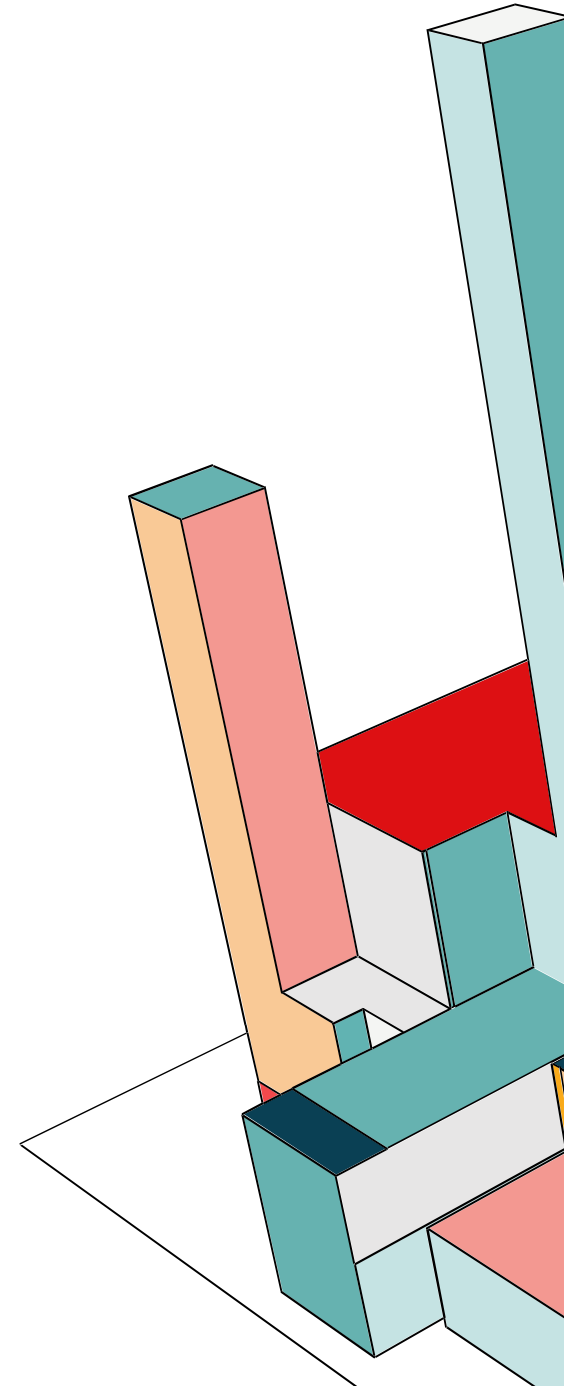
```
def func_void() :  
    print( "Hello!" )
```

```
def func_return() :  
    return 10
```



FUNCTION PARAMETERS & ARGUMENTS

- Parameter variable: variable that is assigned the value of an argument when the function is called
 - The parameter and the argument reference the same value
 - General format:
 - `def function_name( parameter ) :`
 - Scope of a parameter: the function in which the parameter is used

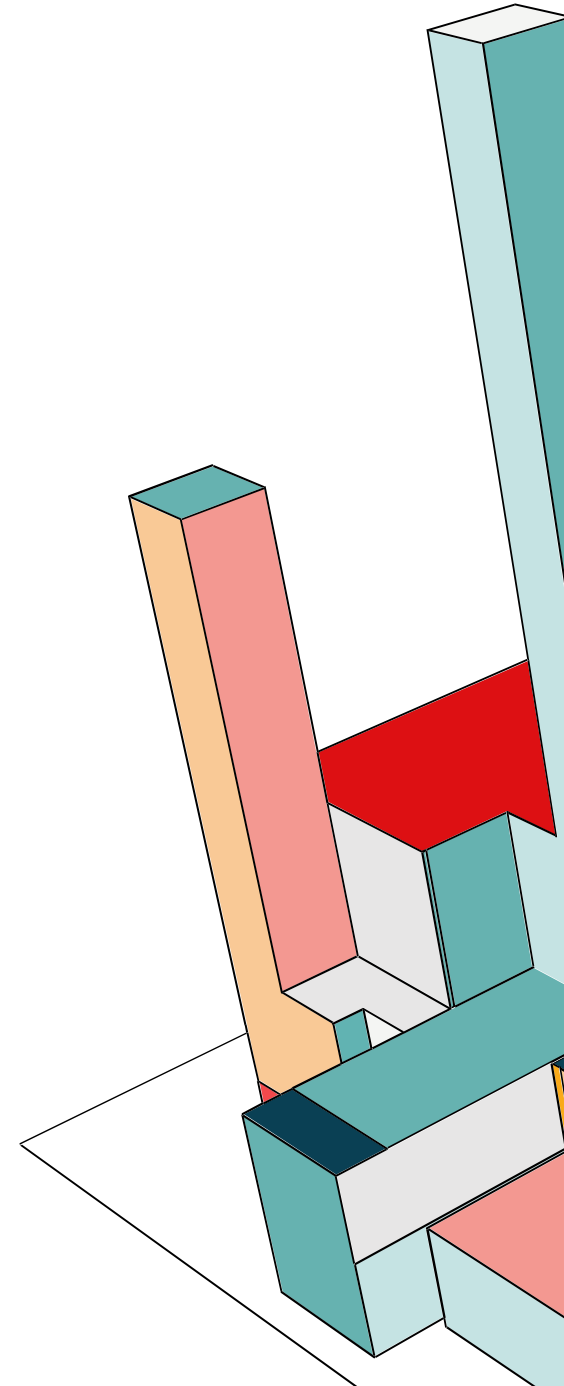


PASSING ARGUMENTS TO FUNCTIONS

```
value = 5  
show_double(value)
```

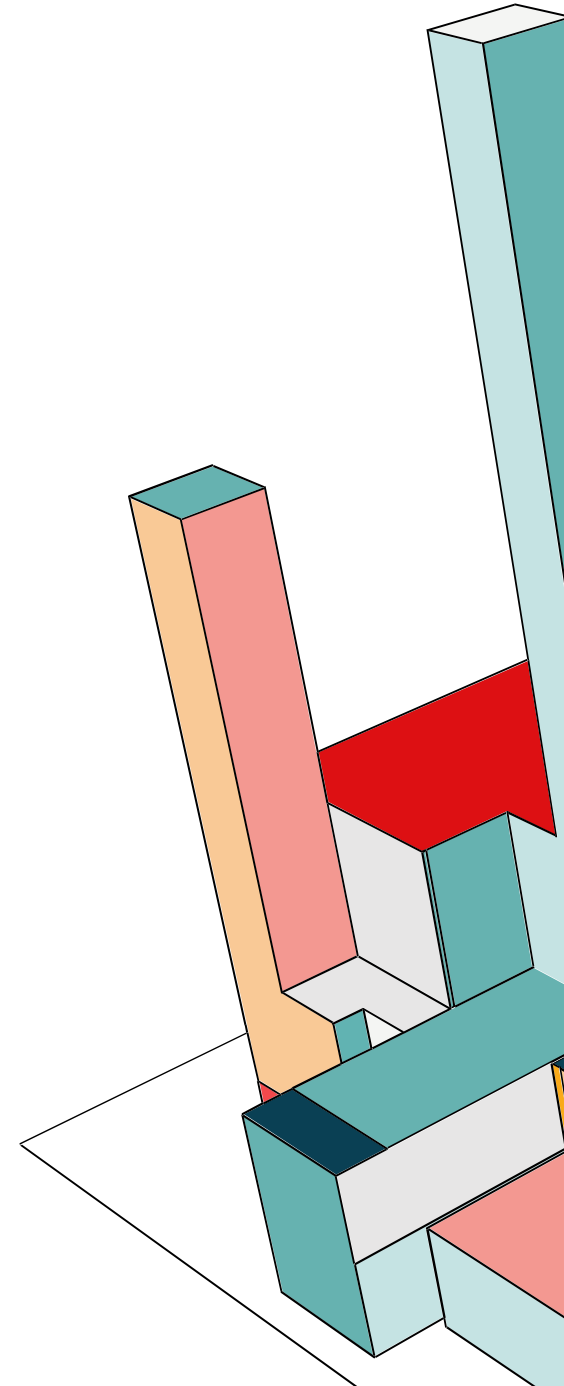


```
def show_double(number):  
    result = number * 2  
    print(result)
```



PASSING MULTIPLE ARGUMENTS

- Python allows writing a function that accepts multiple arguments
 - Parameter list replaces single parameter
 - Parameter list items separated by comma
- Arguments are passed *by position* to corresponding parameters
 - First parameter receives value of first argument, second parameter receives value of second argument, etc.



PASSING MULTIPLE ARGUMENTS

```
show_sum(12, 45)
```

```
def show_sum(num1, num2):  
    result = num1 + num2  
    print(result)
```

